

Front-end herramienta técnica de análisis de resiliencia eléctrica.

Especificación inicial de producto

1. MVP (Producto mínimo viable)

Objetivo del MVP: permitir cargar/seleccionar un escenario, ejecutarlo, entender el resultado y localizar rápidamente dónde se pierde carga, redundancia o capacidad.

Debe incluir:

1. Selector de escenarios

- Escenarios predefinidos.
- Nombre, descripción, severidad esperada.
- Eventos iniciales: fallo EMF, fallo SET, fallo barra 11 kV, fallo trafo, fallo UPS, STS, RMU, sobrecarga, etc.

2. Ejecución de simulación

- Botón “Ejecutar simulación”.
- Estado de ejecución.
- Resultado final:
 - estado global final,
 - tiempo final,
 - carga total,
 - carga servida,
 - carga perdida,
 - porcentaje servido,
 - capacidad entregable,
 - capacidad activa,
 - componentes fallados,
 - reservas en uso.

3. Dashboard de KPIs

- Estado global: operativo / degradado / fallado.
- Carga total vs servida vs perdida.
- Porcentaje de carga servida.
- Capacidad entregable vs demanda.
- Reservas activadas.
- Salas degradadas o sin servicio.
- Componentes fallados.

4. Timeline de eventos

- Tiempo.
- Tipo de evento.
- Objetivo.
- Descripción.
- Estado global antes/después.

- Severidad.
- Eventos derivados: conmutación, entrada de reserva, arranque generador, agotamiento batería, pérdida suministro.

5. **Snapshots**

- Tabla por instante temporal.
- Estado global.
- Carga servida/perdida.
- Capacidad entregable.
- Fallos.
- Reservas en uso.
- Salas degradadas/sin servicio.

6. **Vista de componentes**

- Lista filtrable por tipo:
 - Red, EMF, subestación, transformador, RMU, UPS, STS, busbar, generador, sala, zona.
- Estado: activo, reserva, fallado, mantenimiento, desconectado.
- Criticidad.
- Si es reserva.
- Capacidad nominal.
- Flags especiales: UPS en batería, batería agotada, alimentando zona.

7. **Exportación básica**

- JSON de resultados.
- CSV de eventos.
- CSV de snapshots.

2. **Funcionalidades avanzadas**

Para una segunda fase:

1. **Comparador de escenarios**

- Comparar 2–4 simulaciones.
- Diferencias en carga perdida.
- Tiempo en degradado/fallado.
- Reservas usadas.
- Eventos críticos.
- Primer punto de pérdida de carga.

2. **Vista topológica**

- Grafo eléctrico dirigido.
- Rutas desde fuente hasta zona.
- Colores por estado.
- Destacar ruta activa, ruta fallida y ruta de respaldo.

- Útil porque la topología ya está modelada como grafo dirigido con nodos, conexiones y búsqueda de rutas.

3. **Análisis de causa raíz**

- “Primer evento que causó degradación”.
- “Primer evento que causó pérdida de carga”.
- “Componente que agotó redundancia”.
- “Zona afectada por falta de ruta válida”.

4. **Simulador interactivo**

- Crear eventos manualmente desde UI.
- Fallar componente.
- Recuperar componente.
- Forzar sobrecarga.
- Forzar fallo de conmutación.
- Parar generador.
- Sacar reserva.

5. **Editor de escenarios**

- Formulario para crear escenarios.
- Validación de eventos.
- Guardado de escenarios.
- Duplicar escenario existente.

6. **Playback temporal**

- Slider de tiempo.
- Ver estado del sistema en cada snapshot.
- Avanzar evento a evento.

7. **Heatmap de impacto**

- Salas/zonas con pérdida.
- Componentes con criticidad alta.
- Reservas consumidas.
- Zonas deslastradas.

3. **Tipos de usuario**

Operador técnico

- Ejecuta escenarios.
- Revisa alarmas, eventos y estado final.
- Necesita rapidez y claridad.

Ingeniero eléctrico

- Analiza rutas, redundancia, capacidad y conmutaciones.

- Necesita detalle técnico y trazabilidad.

Responsable de resiliencia / diseño

- Compara escenarios.
- Evalúa si la arquitectura mantiene N+1.
- Necesita KPIs y exportación.

Usuario ejecutivo

- Consulta resumen.
 - Quiere estado global, carga perdida y severidad.
 - No necesita ver todos los eventos técnicos.
-

4. Casos de uso principales

1. Seleccionar escenario predefinido.
 2. Ejecutar simulación.
 3. Ver resultado global.
 4. Identificar si el sistema queda operativo, degradado o fallado.
 5. Revisar carga perdida.
 6. Ver qué reservas entraron en servicio.
 7. Analizar eventos cronológicos.
 8. Revisar snapshots por tiempo.
 9. Ver componentes afectados.
 10. Exportar resultados.
 11. Comparar escenarios.
 12. Crear o modificar escenario.
-

5. Flujo de navegación recomendado

Inicio / Escenarios

- seleccionar escenario
- revisar configuración
- ejecutar simulación
- dashboard de resultados
- timeline / snapshots / componentes / topología
- exportar o comparar

Navegación principal:

- **Escenarios**
- **Resultados**
- **Timeline**
- **Snapshots**
- **Componentes**
- **Topología**
- **Comparador**

- **Exportaciones**
-

6. Información por vista

Vista “Escenarios”

- Nombre.
- Tipo de fallo.
- Componentes afectados.
- Eventos programados.
- Severidad.
- Tiempo de inicio.
- Estado inicial.

Vista “Dashboard”

- Estado global.
- Carga total.
- Carga servida.
- Carga perdida.
- Porcentaje servido.
- Capacidad entregable.
- Reservas en uso.
- Componentes fallados.
- Salas degradadas.
- Salas sin servicio.

Vista “Timeline”

- Tiempo.
- Tipo de evento.
- Descripción.
- Objetivo.
- Severidad.
- Estado antes/después.
- Eventos derivados agrupados.

Vista “Snapshots”

- Tabla temporal.
- Gráfica de carga servida/perdida.
- Gráfica de capacidad entregable.
- Estado global por tramo.

Vista “Componentes”

- ID.
- Nombre.
- Tipo.

- Estado.
- Criticidad.
- Reserva sí/no.
- Capacidad.
- Relación con sala/zona.
- Estado especial según tipo.

Vista “Topología”

- Grafo.
- Nodos por tipo.
- Conexiones activas/falladas.
- Ruta disponible.
- Ruta interrumpida.
- Fuente actual.
- Fuente de respaldo.

Vista “Comparador”

- Escenario A vs B.
- Diferencia de carga perdida.
- Diferencia de reservas usadas.
- Diferencia de eventos críticos.
- Estado final comparado.

7. Acciones del usuario

MVP:

- Seleccionar escenario.
- Ejecutar simulación.
- Filtrar eventos.
- Filtrar componentes.
- Ver detalle de evento.
- Ver detalle de componente.
- Exportar resultados.

Avanzado:

- Crear escenario.
 - Editar eventos.
 - Duplicar escenario.
 - Comparar simulaciones.
 - Reproducir simulación por tiempo.
 - Marcar eventos como críticos.
 - Generar informe.
-

8. Diseño visual general

Estilo recomendado: **industrial, técnico, sobrio.**

- Fondo claro o dark mode técnico.
- Verde: operativo.
- Ámbar: degradado.
- Rojo: fallado / pérdida crítica.
- Azul: reserva / conmutación.
- Gris: desconectado / mantenimiento.
- Tarjetas KPI grandes.
- Tablas densas pero filtrables.
- Timeline horizontal o vertical.
- Grafo topológico como vista avanzada, no como pantalla inicial.

La UI debe priorizar **lectura rápida del impacto**: estado global, carga perdida, reservas usadas y causa principal.

9. Componentes UI necesarios

- ScenarioSelector
 - SimulationRunPanel
 - KpiCard
 - GlobalStatusBadge
 - LoadBalanceChart
 - CapacityChart
 - EventTimeline
 - EventDetailDrawer
 - SnapshotTable
 - SnapshotChart
 - ComponentTable
 - ComponentDetailPanel
 - TopologyGraph
 - ScenarioComparisonTable
 - ExportMenu
 - SeverityBadge
 - StateBadge
 - FilterBar
 - TimeSlider
-

10. Estructura recomendada del front-end

```
src/  
  app/  
    routes/  
      scenarios/  
      results/  
      comparison/  
    features/  
      scenarios/  
      simulation/  
      results/  
      events/  
      snapshots/  
      components/  
      topology/  
      exports/  
    components/  
      ui/  
      charts/  
      tables/  
      layout/  
    domain/  
      types/  
      mappers/  
      constants/  
    services/  
      api/  
      simulationApi.ts  
    store/  
      simulationStore.ts  
    utils/  
      formatters.ts
```

Modelo mental recomendado:

- domain/types: tipos alineados con backend.
 - features/simulation: ejecución y estado actual.
 - features/results: KPIs.
 - features/events: timeline.
 - features/snapshots: evolución temporal.
 - features/components: inventario técnico.
 - features/topology: grafo/rutas.
 - features/exports: descarga.
-

Criterios de aceptación del MVP

1. El usuario puede seleccionar un escenario y ejecutarlo.
 2. La app muestra claramente si el resultado final es operativo, degradado o fallado.
 3. La app muestra carga total, servida, perdida y porcentaje servido.
 4. La app lista todos los eventos procesados en orden cronológico.
 5. La app muestra snapshots por tiempo.
 6. La app permite identificar componentes fallados y reservas activadas.
 7. La app permite identificar salas o zonas sin alimentación.
 8. La app permite exportar resultados.
 9. La UI diferencia visualmente estados eléctricos.
 10. El resultado mostrado coincide con el diccionario devuelto por el motor.
-

Alcance recomendado

MVP:

Dashboard + escenarios + timeline + snapshots + componentes + exportación.

Fase 2:

Topología visual + comparador + análisis causa raíz.

Fase 3:

Editor de escenarios + playback temporal + informes automáticos.

Mi recomendación: diseñar primero el **contrato API frontend-backend** antes que las pantallas, porque las vistas dependen directamente de resultados, registro_eventos, snapshots, componentes, zonas_it, salas_it y topologia.

Resumen:

Requisitos funcionales

Son lo que la aplicación **debe permitir hacer**.

1. Seleccionar un escenario predefinido.
2. Cargar o crear un escenario de simulación.
3. Ejecutar una simulación.
4. Mostrar el estado global final: operativo, degradado o fallado.
5. Mostrar KPIs principales:
 - a. carga total,
 - b. carga servida,
 - c. carga perdida,
 - d. porcentaje servido,
 - e. capacidad entregable,
 - f. capacidad activa,
 - g. reservas en uso,
 - h. componentes fallados.
6. Mostrar el registro cronológico de eventos.
7. Filtrar eventos por tipo, severidad, componente o tiempo.
8. Mostrar snapshots de la simulación.

9. Visualizar la evolución temporal de carga, capacidad y estado.
10. Mostrar el estado de componentes eléctricos.
11. Filtrar componentes por tipo y estado.
12. Identificar componentes fallados.
13. Identificar reservas activadas.
14. Identificar salas o zonas degradadas o sin alimentación.
15. Mostrar pérdida de carga por sala, zona o bloque.
16. Comparar escenarios.
17. Exportar resultados en JSON/CSV.
18. Ver detalle de un evento.
19. Ver detalle de un componente.
20. Visualizar, en fase avanzada, la topología eléctrica y rutas de alimentación.

Requisitos no funcionales

Son cómo debe comportarse la aplicación.

1. **Claridad:** la UI debe permitir entender rápidamente si el sistema está operativo, degradado o fallado.
2. **Rendimiento:** la carga de resultados, eventos y snapshots debe ser fluida incluso con escenarios grandes.
3. **Trazabilidad:** cada KPI debe poder relacionarse con eventos y snapshots.
4. **Consistencia visual:** los estados eléctricos deben usar colores fijos:
 - a. verde: operativo/activo,
 - b. ámbar: degradado/reserva usada,
 - c. rojo: fallado/pérdida,
 - d. gris: desconectado/mantenimiento.
5. **Fiabilidad:** los datos mostrados deben coincidir con la salida del motor de simulación.
6. **Usabilidad:** navegación simple: escenarios → ejecución → resultados → análisis.
7. **Escalabilidad:** la estructura debe permitir añadir comparador, topología, editor de escenarios e informes.
8. **Mantenibilidad:** el front-end debe estar organizado por dominios: escenarios, simulación, eventos, snapshots, componentes, topología.
9. **Auditabilidad:** el usuario debe poder reconstruir qué ocurrió, cuándo ocurrió y qué impacto tuvo.
10. **Exportabilidad:** los resultados deben poder descargarse para análisis externo o reporting.
11. **Robustez:** la aplicación debe gestionar errores del backend, escenarios inválidos o simulaciones fallidas.
12. **Precisión técnica:** unidades, estados y nombres de componentes deben mostrarse de forma coherente.
13. **Responsividad:** debe funcionar correctamente en escritorio y pantallas grandes; móvil no es prioritario.
14. **Seguridad:** si hay usuarios y escenarios guardados, debe existir control de acceso.
15. **Extensibilidad:** debe permitir incorporar nuevos tipos de eventos, componentes y métricas sin rediseñar toda la UI.

Stack recomendado

Backend Python

- **FastAPI** para exponer el motor como API REST. Encaja muy bien con Python, tipado y modelos Pydantic, y genera documentación OpenAPI automáticamente. ([FastAPI](#))
- Tu motor actual seguiría igual: MotorSimulacion, MotorReglas, escenarios y modelos.

Frontend

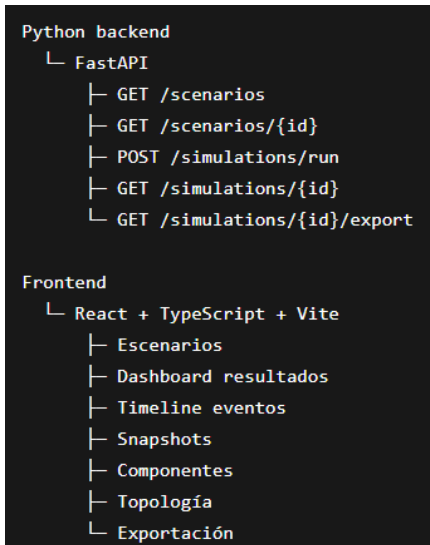
- **React + TypeScript.**
- Para MVP, usaría **Vite** antes que Next.js: más simple, rápido y suficiente para una app interna/técnica tipo dashboard. Vite está pensado para proyectos web modernos con desarrollo rápido. ([vitejs](#))
- **Next.js** tendría sentido si luego quieres autenticación avanzada, SSR, rutas complejas o despliegue tipo producto SaaS. ([Next.js](#))

UI

- **Tailwind CSS** para estilos.
- **shadcn/ui** para componentes profesionales: cards, tablas, tabs, dialogs.
- **Recharts** para gráficas de carga/capacidad.
- **TanStack Table** para tablas potentes.

- **React Flow** para la topología eléctrica, porque está pensado para interfaces de nodos, diagramas y flujos. ([React Flow](#))

Arquitectura propuesta



Contrato API mínimo

El backend debería devolver algo así:

```

</> JSON

{
  "scenario_id": "dc1_fallo_emf",
  "estado_global_final": "degradado",
  "tiempo_final_s": 100,
  "kpis": {
    "carga_total_kw": 32400,
    "carga_servida_kw": 32400,
    "carga_perdida_kw": 0,
    "porcentaje_carga_servida": 100,
    "capacidad_entregable_kw": 28000,
    "capacidad_total_activa_kw": 30000,
    "num_componentes_fallados": 1,
    "num_reservas_en_uso": 3
  },
  "registro_eventos": [],
  "snapshots": [],
  "componentes": [],
  "salas_it": [],
  "zonas_it": [],
  "topologia": {
    "nodos": [],
    "conexiones": []
  }
}
  
```

Camino de implementación

Fase 1 — API

- Envolver tu motor Python con FastAPI.
- Crear endpoint GET /scenarios.
- Crear endpoint POST /simulations/run.
- Normalizar la salida del motor.

Fase 2 — Frontend MVP

- Pantalla de escenarios.
- Botón ejecutar.
- Dashboard KPIs.
- Timeline.
- Tabla de snapshots.
- Tabla de componentes.

Fase 3 — Análisis avanzado

- Comparador de escenarios.

- Topología con React Flow.
- Playback temporal.
- Exportación avanzada.
- Editor de escenarios.

Recomendación concreta

Para empezar:

FastAPI + React + TypeScript + Vite + Tailwind + shadcn/ui + Recharts + TanStack Table

Es el stack más equilibrado: simple, moderno, técnico y muy compatible con tu backend Python.